

Python Function (Module-5)

Function

A function is a collection of related assertions that performs a mathematical, analytical, or evaluative operation. Python functions are simple to define and essential to intermediate-level programming.

Function vs Method

- A function doesn't need any object and is independent, while the method is a function, which is linked with any object.
- We can directly call the function with its name, while the method is called by the object's name.
- Function is used to pass or return the data, while the method operates the data in a class.
- Function is an independent functionality, while the method lies under object-oriented programming.
- In functions, we don't need to declare the class, while to use methods we need to declare the class.
- Functions can only work with the provided data, while methods can access all the data provided in the given class.

Python math Module

Python math module is defined as the most famous mathematical functions, which includes trigonometric functions, representation functions, logarithmic functions, etc. Furthermore, it also defines two mathematical constants, i.e., Pie and Euler number, etc.

Use a Module

We can use the module by using the **import** statement:

Example

```
import math
x=math.sqrt(4)
```

Re-naming a Module

You can create an alias when you import a module, by using the **as** keyword

```
import math as mt
x=mt.sqrt(4)
```

Import from Module

You can choose to import only parts from a module, by using the **from** keyword

```
from math import sqrt
x=sqrt(4)
```

'math' module methods

math.ceil(x)

This method returns the ceil value of the x. It returns the greater than or equal value to x.

```
import math
number = math.ceil(10.25201)
print("The ceil value is:",number)
```

Output:

The ceil value is: 11

math.pow(x,y)

This method returns the power of the x corresponding to the value of y. If value of x is negative or y is not integer value than it raises a **ValueError**.

Example

```
import math
number = math.pow(10,2)
print("The power of number:",number)
```

Output:

The power of number: 100.0

math.floor(x)

This method returns the floor value of the x. It returns the less than or equal value to x.

Example:

```
import math
number = math.floor(10.25201)
print("The floor value is:",number)
```

Output:

The floor value is: 10

math.fabs(x)

This method returns the absolute value of x.

Example:

```
import math
number = math.fabs(10.001)
print("The absolute is:",number)
```

Output:

The absolute value is: 10.001

math.factorial()

This method returns the factorial of the given number x. If x is not integral, it raises a **ValueError**.

Example

```
import math
number = math.factorial(7)
print("The factorial of number:",number)
```

Output:

The factorial of number: 5040

math.modf(x)

This method returns the fractional and integer parts of x. It carries the sign of x.

Example

```
import math
number = math.modf(44.5)
print("The modf of number:", number)
```

Output:

The modf of number: (0.5, 44.0)

math.copysign()

Return the value of the first parameter and the sign of the second parameter.

Example

```
import math
print(math.copysign(4, -1))
```

Output:

-4.0

math.fsum()

Return the sum of all items in a sequence.

Example

```
import math
print(math.fsum([1, 2, 3, 4, 5]))
```

Output:

15

math.gcd()

Find the greatest common divisor of the two integers.

Example

```
import math
print (math.gcd(3, 6))
```

Output:

3

math.isfinite()

Check whether a value is finite or not.

Example

```
import math
print(math.isfinite(2000))
print(math.isfinite(math.inf))
```

Output:

True
False

math.isinf()

Check whether a value is infinity or not.

Example

```
import math
print(math.isinf(45.34))
print(math.isinf(math.inf))
```

Output:

False
True

math.isnan()

Check whether a value is NaN (Not a Number) or not.

Example

```
import math
print (math.isnan (math.inf))
print (math.isnan (float("nan")))
```

Output:

False
True

math.isqrt()

Round a square root number downwards to the nearest integer.

Example

```
import math
print (math.isqrt(10))
```

Output:

3

math.trunc()

Find the integer part of different numbers.

Example

```
import math
print (math.trunc(3.2))
```

Output:

3

Function Composition in Python

Function composition is the way of combining two or more functions in such a way that the output of one function becomes the input of the second function and so on. For example, let there be two functions “F” and “G” and their composition can be represented as $F(G(x))$ where “x” is the argument and output of G(x) function will become the input of F() function.

Example

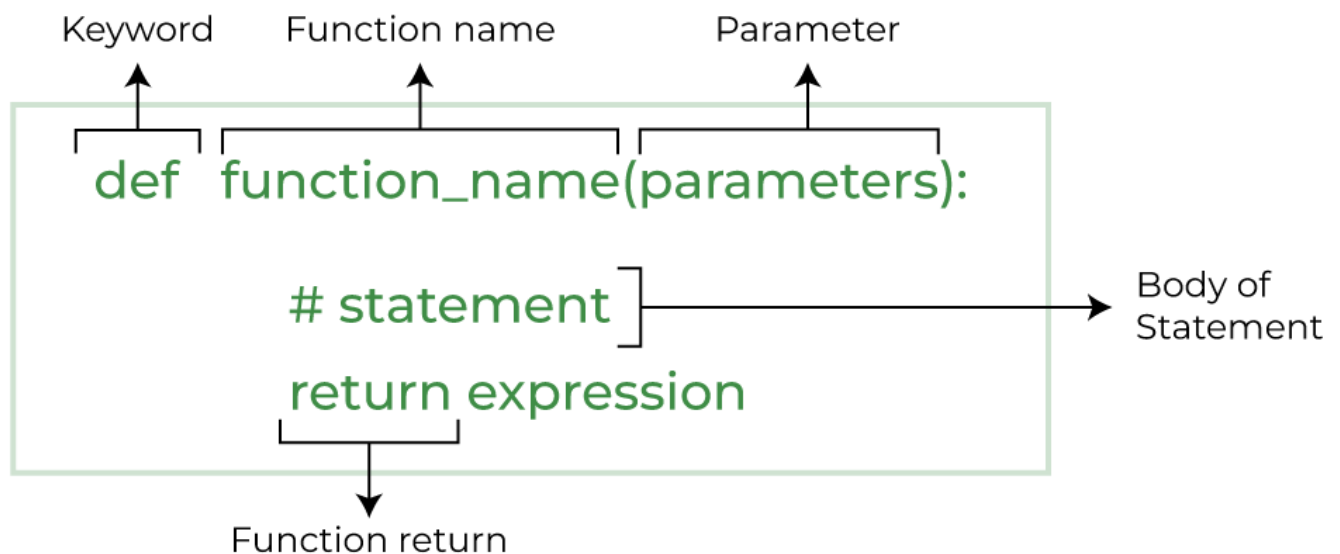
```
import math
x = math.sqrt(math.ceil(3.4))
print(x)
```

Output

2.0

Here the output of the ceil() function becomes the input of the sqrt() function.

Defining our own functions (User defined functions)



Example:

Definition of function

```
def sum():
    a = int(input("Enter 1st number:"))
    b = int(input("Enter 1st number:"))
    c = a+b
    print(c)
```

Calling

```
sum()
```

Output:

```
Enter 1st number: 10
Enter 1st number: 20
30
```

```
# Definition of function
def sum(a, b):
    c = a+b
    print(c)
# Calling
sum(10, 20)
```

Output:

30

```
# Definition of function
def sum(a, b):
    c = a+b
    return c
# Calling
x = sum(10, 20)
print(x)
```

Output:

30

Types of Arguments

Python supports various types of arguments that can be passed at the time of the function call. Let's discuss each type in detail.

Formal Parameter

Parameter Written in Function Definition is Called "Formal Parameter. Formal parameters are always variables.

Actual Parameter

Parameter Written in Function Call is Called "Actual Parameter". One can use numbers, expressions, or even function calls as actual parameters.

Default arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument. **We must specify the default arguments from the right side.** The following example illustrates Default arguments.

```
def add(a, b, c=10):
    return (a+b+c)

print(add(5, 7, 15))
print(add(20, 50)) # taking default value of c=10
```

Output:

27
80

Positional arguments

Arguments are passed in the order of parameters. The order defined in the order function declaration.

Keyword arguments

The idea is to allow the caller to specify the argument name with values so that caller does not need to remember the order of parameters.

```
def show(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
show(40, "Anindya Sen")  
show(age=50, name="Subrata De")
```

Output:

Name: 40
Age: Anindya Sen

Name: Subrata De
Age: 50

Variable-length arguments

In Python, we can pass a variable number of arguments to a function using special symbols. There are two special symbols:

1. `*args` (Non-Keyword Arguments)
2. `**kwargs` (Keyword Arguments)

Variable length non-keywords argument

```
def show(*args):  
    for x in args:  
        print(x)  
  
show("hello", 20, 30, "good")
```

Output:

hello
20
30
good

Variable length keywords argument

```
def show(**args):  
    for x in args.values():  
        print(x)  
show(first="Sanjoy", middle="Kumar", last="Das")
```

Output:

```
Sanjoy  
Kumar  
Das
```

```
def show(**args):  
    for x, y in args.items():  
        print(x, "=", y)  
  
show(first="Sanjoy", middle="Kumar", last="Das")
```

Output:

```
first = Sanjoy  
middle = Kumar  
last = Das
```

Returning Multiple Values from a function

1. Using Tuple: We can return multiple values from a function using a tuple.

```
def addsub(a, b):  
    return a + b, a - b  
  
x, y = addsub(10, 5)  
print(f"Addition={x} and Substraction={y}")
```

Output

```
Addition=15 and Substraction=5
```

2. Using List: We can return multiple values from a function using a tuple.

```
def addsub(a, b):  
    return [a + b, a - b]  
  
x, y = addsub(10, 5)  
print(f"Addition={x} and Substraction={y}")
```

Output

```
Addition=15 and Substraction=5
```


Docstring

The first string after the function is called the Document string or Docstring in short. This is used to describe the functionality of the function. The use of docstring in functions is optional but it is considered a good practice.

The below syntax can be used to print out the docstring of a function:

Syntax: `print(function_name.__doc__)`

```
def add(a, b):  
    '''This function adds two numbers'''  
    return (a+b)
```

```
print(add.__doc__)
```

Output:

This function adds two numbers

Pass by Reference or pass by value

One important thing to note is, in Python every variable name is a reference. When we pass a variable to a function, a new reference to the object is created. Parameter passing in Python is the same as reference passing in Java.

```
def myFun(x):  
    x[0] = 20  
  
lst = [10, 11, 12, 13, 14, 15]  
myFun(lst)  
print(lst)
```

Output:

[20, 11, 12, 13, 14, 15]

When we pass a reference and change the received reference to something else, the connection between the passed and received parameter is broken. For example, consider the below program as follows:

```
def myFun(x):  
    x = [20, 30, 10]  
  
lst = [10, 11, 12, 13, 14, 15]  
myFun(lst)  
print(lst)
```

Output:

[10, 11, 12, 13, 14, 15]

Local & Global Variable

Python Global variables are those which are not defined inside any function and have a global scope whereas Python **local variables** are those which are defined inside a function and their scope is limited to that function only. In other words, we can say that local variables are accessible only inside the function in which it was initialized whereas the global variables are accessible throughout the program and inside every function.

Example

```
x = 10
def show():
    y = 20
    print("x=", x)
    print("y=", y)
show()
print("x=", x)
```

Output:

```
x= 10
y= 20
x= 10
```

In the above example 'x' is the global variable and 'y' is the local variable.

Recursion

What is Recursion?

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily.

Need of Recursion

Recursion is an amazing technique with the help of which we can reduce the length of our code and make it easier to read and write. It has certain advantages over the iteration technique which will be discussed later. A task that can be defined with its similar subtask, recursion is one of the best solutions for it. For example; The Factorial of a number.

Properties of Recursion:

- Performing the same operations multiple times with different inputs.
- In every step, we try smaller inputs to make the problem smaller.
- Base condition is needed to stop the recursion otherwise infinite loop will occur.

Advantages of using recursion

- A complicated function can be split down into smaller sub-problems utilizing recursion.
- Sequence creation is simpler through recursion than utilizing any nested iteration.
- Recursive functions render the code look simple and effective.

Disadvantages of using recursion

- A lot of memory and time is taken through recursive calls which makes it expensive for use.
- Recursive functions are challenging to debug.
- The reasoning behind recursion can sometimes be tough to think through.

How are recursive functions stored in memory?

Recursion uses more memory, because the recursive function adds to the stack with each recursive call, and keeps the values there until the call is finished. The recursive function uses LIFO (LAST IN FIRST OUT) Structure just like the stack data structure.

What is the base condition in recursion?

In the recursive program, the solution to the base case is provided and the solution to the bigger problem is expressed in terms of smaller problems.

```
def fact(n):  
    if n <= 1: # base case  
        return 1  
    else:  
        return n*fact(n-1)
```

In the above example, the base case for $n \leq 1$ is defined and the larger value of a number can be solved by converting to a smaller one till the base case is reached.

How a particular problem is solved using recursion?

The idea is to represent a problem in terms of one or more smaller problems, and add one or more base conditions that stop the recursion. For example, we compute factorial n if we know the factorial of $(n-1)$. The base case for factorial would be $n = 0$. We return 1 when $n = 0$.

Why Stack Overflow error occurs in recursion?

If the base case is not reached or not defined, then the stack overflow problem may arise. Let us take an example to understand this.

```
def fact(n):  
    # wrong base case (it may cause stack overflow).  
    if n == 100:  
        return 1  
  
    else:  
        return n*fact(n-1)
```

If `fact(10)` is called, it will call `fact(9)`, `fact(8)`, `fact(7)`, and so on but the number will never reach 100. So, the base case is not reached. If the memory is exhausted by these functions on the stack, it will cause a stack overflow error.

What is the difference between direct and indirect recursion?

A function fun is called direct recursive if it calls the same function fun. A function fun is called indirect recursive if it calls another function say fun_new and fun_new calls fun directly or indirectly. The difference between direct and indirect recursion has been illustrated in Table 1.

#An example of direct recursion

```
def directRecFun():
```

```
    # Some code....
```

```
    directRecFun()
```

```
    # Some code...
```

An example of indirect recursion

```
def indirectRecFun1():
```

```
    # Some code...
```

```
    indirectRecFun2()
```

```
    # Some code...
```

```
def indirectRecFun2():
```

```
    # Some code...
```

```
    indirectRecFun1()
```

```
    # Some code...
```

What is Tail-Recursion?

A unique type of recursion where the last procedure of a function is a recursive call. The recursion may be automated away by performing the request in the current stack frame and returning the output instead of generating a new stack frame. The tail-recursion may be optimized by the compiler which makes it better than non-tail recursive functions.

Is it possible to optimize a program by making use of a tail-recursive function instead of non-tail recursive function?

Considering the function given below in order to calculate the factorial of n, we can observe that the function looks like a tail-recursive at first but it is a non-tail-recursive function. If we observe closely, we can see that the value returned by `Recur_facto(n-1)` is used in `Recur_facto(n)`, so the call to `Recur_facto(n-1)` is not the last thing done by `Recur_facto(n)`.

Program to calculate factorial of a number

using a Non-Tail-Recursive function.

non-tail recursive function

```
def Recur_facto(n):
```

```
    if (n == 0):
```

```
        return 1
```

```
    return n * Recur_facto(n-1)
```

```
# print the result
print(Recur_facto(6))
```

```
# Program to calculate factorial of a number
# using a Tail-Recursive function.
```

```
# A tail recursive function
def Recur_facto(n, a = 1):

    if (n == 0):
        return a

    return Recur_facto(n - 1, n * a)
```

```
# print the result
print(Recur_facto(6))
```

Python Lambda Functions

Python Lambda Functions are anonymous function means that the function is without a name. As we already know that the `def` keyword is used to define a normal function in Python. Similarly, the `lambda` keyword is used to define an anonymous function in Python.

Syntax: *lambda arguments: expression*

- This function can have any number of arguments but only one expression, which is evaluated and returned.
- One is free to use lambda functions wherever function objects are required.
- You need to keep in your knowledge that lambda functions are syntactically restricted to a single expression.
- It has various uses in particular fields of programming, besides other types of expressions in functions.

```
sum=lambda x,y:x+y
print(sum(10,20))
```

Output:
30

Python Lambda Function with if-else

Here we are using the **Max** lambda function to find the maximum of two integers.

```
max = lambda a, b : a if(a > b) else b
print(max(1, 2))
```

Using lambda() Function with filter()

The `filter()` function in Python takes in a function and a list as arguments. This offers an elegant way to filter out all the elements of a sequence “sequence”, for which the function returns True. Here is a small program that returns the odd numbers from an input list:

Filter out all even numbers using filter() and lambda function:

Here, lambda x: (x % 2 == 0) returns True or False if x is not even. Since filter() only keeps elements where it produces **True**, thus it removes all odd numbers that generated **False**.

```
li = [5, 7, 22, 97, 54, 62, 77, 23, 73, 61]
final_list = list(filter(lambda x: (x % 2 == 0), li))
print(final_list)
```

Output:

```
[22, 54, 62]
```

Using lambda() Function with map()

The map() function in Python takes in a function and a list as an argument. The function is called with a lambda function and a list and a new list is returned which contains all the lambda-modified items returned by that function for each item. Example:

Multiply all elements of a list by 2 using lambda and map() function:

The code doubles each element in a list using a lambda function and the 'map' function. It then prints the new list with the doubled elements. The output displays each element from the original list, multiplied by 2.

```
li = [1, 2, 3, 4, 5]
```

```
final_list = list(map(lambda x: x*2, li))
print(final_list)
```

Output:

```
[2, 4, 6, 8, 10]
```

What is a pip?

Python pip (Preferred Installer Program) is the package manager for Python packages. We can use pip to install packages that do not come with Python.

Check PIP version:

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip --version
```

Download a package named "playsound":

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip install playsound
```

```
import playsound
playsound.playsound("D:\Python_Program_BCA21\sound.wav")
```

Use the uninstall command to remove a package:

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>pip uninstall
pLaysound
```